

电商购物平台——订单与商品管理系统

系统设计与实现文档

指导教师: 袁时金

完成日期: 2026 年 8 月

项目成员

2451883 孙铭坤 (组长)	2451917 秦康乔
2352164 宋金昊	2452606 陈祉越
2450204 郑书宏	2452688 余伟强
2451389 颜泽霖	2454339 张一诺
2451460 庞乐鸣	2557326 庄辰和

同济大学计算机科学与技术学院

目 录

1 系统需求	1
1.1 系统功能性需求	1
1.2 系统非功能性需求	2
1.3 组织结构	2
2 系统设计与实现	2
2.1 系统总体架构	2
2.2 用户注册与登录用例	3
2.2.1 用例设计	3
2.2.2 用例实现	4
2.3 商品浏览与搜索用例	5
2.3.1 用例设计	5
2.3.2 用例实现	6
2.4 购物车管理用例	7
2.4.1 用例设计	7
2.4.2 用例实现	8
2.5 收货地址管理用例	9
2.5.1 用例设计	9
2.5.2 用例实现	9
2.6 订单创建用例	10
2.6.1 用例设计	10
2.6.2 用例实现	10
2.7 订单支付用例	12
2.7.1 用例设计	12
2.7.2 用例实现	13
2.8 订单管理与超时关闭用例	14
2.8.1 用例设计	14
2.8.2 用例实现	14
2.9 优惠券领取与使用用例	16
2.9.1 用例设计	16
2.9.2 用例实现	16
2.10 物流发货与轨迹用例	17
2.10.1 用例设计	17
2.10.2 用例实现	17
2.11 商品评价用例	18
2.11.1 用例设计	18
2.11.2 用例实现	19
2.12 后台商品与库存管理用例	20
2.12.1 用例设计	20
2.12.2 用例实现	20
2.13 后台权限、统计与审计用例	22
2.13.1 用例设计	22
2.13.2 用例实现	22

3 数据库设计	24
3.1 E-R 图	24
3.2 关系图	25
3.3 关系模式说明	26
A 图表索引	28

1 系统需求

1.1 系统功能性需求

本系统是一个面向中小企业的自营 B2C 电商购物平台，由平台管理员统一维护商品与后台运营，普通用户完成从注册登录、浏览选购到下单支付、收货评价的完整购物流程。系统按业务域划分为用户与权限、商品与库存、购物车与订单、支付与履约、营销与统计五大模块，共 21 项功能需求（其中物流信息录入与物流轨迹维护拆分为两项），具体如下。

(1) 用户与权限模块

- 用户注册、登录与登出：支持用户名、密码注册，密码以 PBKDF2 哈希存储；登录后以 Cookie 维持会话，会话带角色信息。
- 权限分级管理：系统设置普通用户（USER）、客服（SERVICE）、管理员（ADMIN）三级角色，通过“角色—权限”关联实现路径级、操作级权限控制，并支持动态绑定。
- 多收货地址管理：支持地址的增删改查、默认地址设置，一个用户至多一条有效默认地址，采用软删除。
- 后台操作日志审计：记录后台关键写操作（操作人、模块、动作、IP、请求参数、结果），便于追溯与审计。

(2) 商品与库存模块

- 商品分类管理：支持多层级树形分类（父子分类自引用，最多五层），可增改、删除或禁用。
- 商品信息管理：商品支持上下架/预售状态、主图与多图、规格（名称—值）定义，以及复合保存（商品、图片、规格、SKU 一次提交）。
- SKU 与库存管理：SKU 由规格组合唯一确定，维护实际库存与锁定库存；支持库存手工调整、低库存预警与库存变动流水查询。
- 库存自动扣减与回滚：下单锁定库存、支付扣减、取消或超时释放，可用库存 = 实际库存 - 锁定库存，防止超卖。
- 商品热度与推荐：统计商品浏览量、累计销量、评分均值，按热度排序并提供相关推荐。
- 商品评价管理：用户对已完成订单中的商品发表评价（评分、图文、匿名），管理员审核发布或屏蔽。

(3) 购物车与订单模块

- 购物车管理：购物车项以 SKU 为单位，支持添加、修改数量、勾选/取消勾选、删除、清空，结算仅统计勾选项。
- 订单创建、取消与确认收货：支持下单预览（金额试算）、正式创建、取消订单、确认收货，订单明细与收货人采用下单时快照。
- 订单状态流转：待支付（0）→ 已支付（1）→ 已发货（2）→ 已完成（3），另有已取消（4）；状态流转受数据库触发器强约束。
- 订单分页与条件查询：支持“我的订单”与后台订单的分页查询、状态/时间/订单号等条件筛选。
- 订单超时自动关闭：待支付订单超过 30 分钟未支付，由后台定时任务自动取消并释放锁定库存。

(4) 支付与履约模块

- 持久化模拟支付：系统采用模拟支付通道，真实写入支付记录；支付回调采用 HMAC-SHA256 签名校验，幂等处理。
- 物流信息录入与查询：客服对已支付订单发货（填写物流公司与运单号），一个订单对应一条

物流记录。

- 物流轨迹维护：支持按时间追加物流轨迹（已揽收、运输中、派送中、已签收状态推进，不可回退），用户可查询订单物流。

(5) 营销与统计模块

- 优惠券发放、领取与核销：支持满减券与折扣券两种类型，后台创建模板，用户领取，下单时按规则校验并原子核销，一单最多一张。

- 订单统计：提供工作台汇总（今日订单、待发货、今日销售额、库存预警、待审评价）、按日/月维度的订单统计与热销商品榜。

- 数据导出：支持订单报表与库存流水导出为 Excel 文件（页面上限 5000 行）。

1.2 系统非功能性需求

- **安全性**：用户密码采用 PBKDF2（100 000 次迭代，SHA-256）加盐哈希存储，登录态以 Cookie 维持并由后台定时重验用户状态与角色；模拟支付回调通过 HMAC-SHA256 签名与恒定时间比较验签；数据库连接串与回调密钥通过配置注入（配置文件中仅含占位值，生产环境由环境变量覆盖，不提交真实密码）。

- **并发与一致性**：核心并发场景（库存扣减、优惠券领取/核销、支付幂等、订单状态流转、购物车加购）均采用“单条条件 UPDATE + 影响行数判定”的原子操作，配合 Oracle 行级锁与数据库唯一约束/检查约束，防止超卖、重复领取与状态并发冲突。

- **数据完整性**：通过外键（31 条）、检查约束（金额勾稽、库存非负、数量为正、状态取值合法）、函数唯一索引（每用户一个默认地址）、数据库触发器（订单状态流转、支付金额一致、地址归属校验、支付后销量累加）在数据库层保障数据一致性。

- **性能**：列表查询采用分页与多列组合索引；订单统计使用“快照表 + 定时回填”避免实时聚合大表；订单导出限制行数上限，防止内存溢出。

- **可审计性**：订单状态变更写订单日志（from/to 状态、操作人），库存变动写库存流水（变动前后库存），后台关键写操作写操作日志，三者均可追溯。

- **可用性与运维**：支持一键部署与健康检查（/health/ready 探测数据库连通性）；后台统计快照任务与订单超时任务失败可重试，不影响主业务流程。

1.3 组织结构

本文档的组织结构如下。

第一章：系统需求。对系统的功能性与非功能性需求进行说明。

第二章：系统设计与实现。以用例为单位，对每个用例的用例设计（功能设计、动作序列、重要类及方法）与用例实现（关键代码、界面截图）进行说明。

第三章：数据库设计。给出系统的 E-R 图、数据库关系图与关系模式说明。

附录 A：本文档的图表索引。

2 系统设计与实现

2.1 系统总体架构

系统采用前后端分离的 B/S 架构，后端为 ASP.NET Core MVC (.NET 8)，数据库为 Oracle 18c。解决方案按职责划分为五个层次，依赖方向严格单向：ECommerce.Web（表现层）依赖

Application、Infrastructure 与 Shared；Application（应用接口层）只定义服务接口与 DTO，不写 SQL；Infrastructure（基础设施层）实现服务与仓储，使用 Oracle 连接与事务；Domain（领域层）与 Shared（共享契约层）为两个独立的最低层类库。请求流转为：浏览器 → Web 控制器 → 服务接口 → 服务实现 → 仓储 → Oracle，返回逆序。系统分层架构如图 2-1 所示。

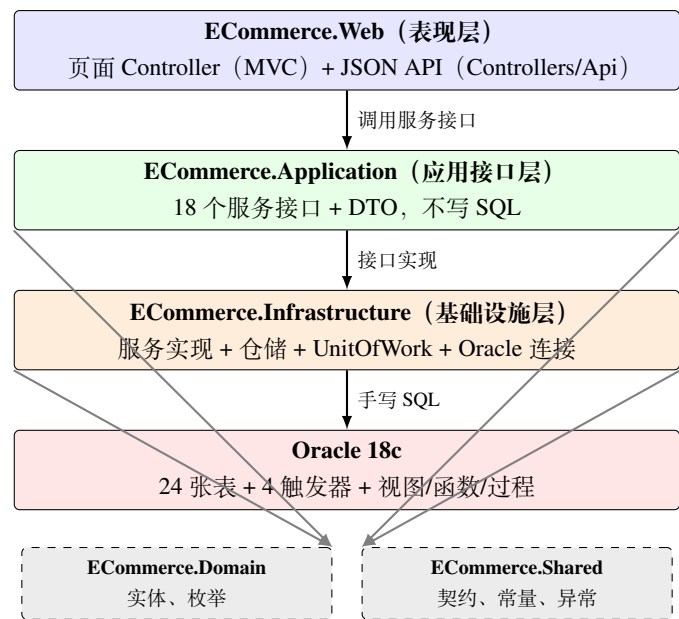


图 2-1 系统分层架构与请求流转（实线为主调用链，虚线为 Domain/Shared 被上层引用）

数据访问不使用 ORM，而是基于 Oracle 原生 SQL 的手写仓储（UnitOfWork + OracleConnectionFactory）。注册、下单、支付、发货、取消、默认地址设置等关键写操作，每个操作均在一个数据库事务内完成，失败整体回滚。认证采用 Cookie + Claims，授权采用三层模型（端点级策略、动态 RBAC 过滤器、视图角色渲染）。以下按用例对系统设计与实现进行说明，每个用例含用例设计与用例实现两个部分。文中引用的代码片段均为节选，省略了参数校验、样板代码等非核心逻辑，核心业务逻辑与源码保持一致。

2.2 用户注册与登录用例

2.2.1 用例设计

用户注册与登录用例面向普通用户。用户注册时填写用户名、密码、手机号、邮箱，系统校验用户名唯一性与密码长度（不少于 6 位），将密码以 PBKDF2（100 000 次迭代、SHA-256、随机盐）哈希后入库，并在同一事务内为用户绑定默认角色 USER。用户登录时校验密码哈希与账号状态，成功后由 Web 层写入 Cookie（含用户 ID、用户名、角色集合），登录态通过滑动过期与后台定时重验维持。动作序列如表 2-1 所示。

表 2-1 用户注册与登录用例的动作序列

动作序列	描述
用户注册	校验用户名唯一、密码长度；PBKDF2 哈希；事务内创建用户并绑定 USER 角色。
注册成功	自动登录，写入会话 Cookie，跳转首页。
用户登录	按用户名查用户，恒定时间比较密码哈希，校验账号启用状态。
写入会话	更新最后登录时间，查询角色，由 Web 层签名 Cookie。
会话维持	Cookie 滑动过期；定时重验用户存在与启用状态并同步角色。

本用例涉及的重要类与方法: 服务接口 `IAuthService` (`RegisterAsync`、`LoginAsync`)、服务实现 `AuthService`、仓储 `IUserRepository` (`GetByUsernameAsync`、`CreateUserWithDefaultRoleAsync`、`GetRoleNamesAsync`)、安全组件 `PasswordHashing` (PBKDF2 哈希与恒定时间校验)、`RefreshUserPrincipalCookieEvents` (会话保鲜与角色同步)。

2.2.2 用例实现

`AuthService.LoginAsync` 的核心实现如代码 2-1 所示: 按用户名查库, 用 `PasswordHashing.Verify` 进行恒定时间密码比较, 校验账号状态, 更新最后登录时间并返回带角色的会话 DTO。

```
1 public async Task<UserSessionDto> LoginAsync(LoginRequest request, CancellationToken ct =
   default)
2 {
3     var username = NormalizeRequired(request.Username, "用户名不能为空");
4     var password = NormalizeRequired(request.Password, "密码不能为空");

6     var user = await _userRepository.GetByUsernameAsync(username, ct);
7     if (user is null || !PasswordHashing.Verify(password, user.PasswordHash))
8     {
9         throw new BusinessException(ErrorCodes.AuthInvalidCredentials, "用户名或密码错误");
10    }
11    if (user.Status == 0)
12    {
13        throw new BusinessException(ErrorCodes.UserDisabled, "账号已被禁用");
14    }

16    await _userRepository.UpdateLastLoginAsync(user.Id, ct);
17    var roles = await _userRepository.GetRoleNamesAsync(user.Id, ct);
18    return new UserSessionDto(user.Id, user.Username, roles);
19 }
```

代码 2-1 `AuthService.LoginAsync` 登录校验

登录成功后, 由 Web 层 `AccountController` 调用 `SignInAsync` 写入 Cookie (含 `NameIdentifier`、`Name`、每个角色一个 `Role` 声明), 并依据角色跳转: `ADMIN/SERVICE` 进入后台 `/admin`, 普通用户进入首页。登录页面与注册页面分别如图 2-2、图 2-3 所示。

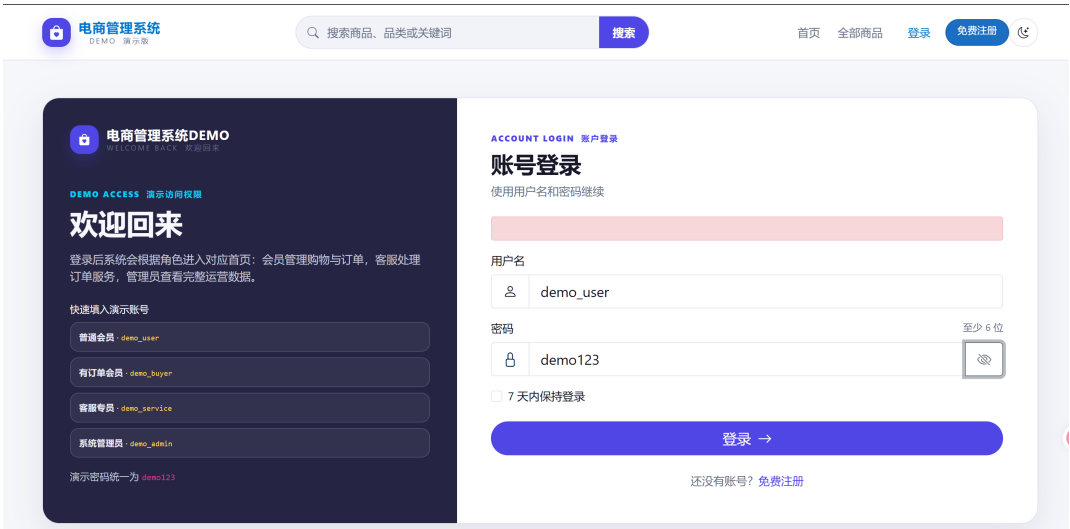


图 2-2 用户登录页面

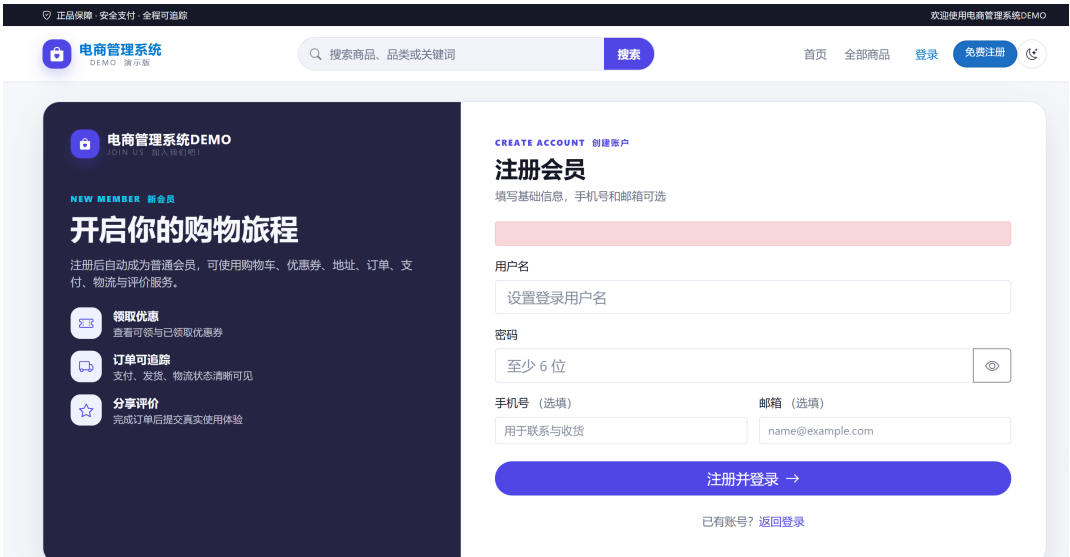


图 2-3 用户注册页面

2.3 商品浏览与搜索用例

2.3.1 用例设计

商品浏览与搜索用例面向所有访问者，无需登录。系统提供分类树、商品列表、商品详情与相关推荐四类能力：分类树支持多层级（最多五层）；商品列表支持按分类、关键词、排序方式（热度/销量/评分/价格升序/降序）分页查询，前台仅展示上架或预售商品，并按所选分类递归包含全部子分类商品；商品详情在展示商品、图片、规格、SKU 与评价的同时累加浏览量。动作序列如表 2-2 所示。

表 2-2 商品浏览与搜索用例的动作序列

动作序列	描述
加载分类树	查询启用状态的分类并按层级、排序号构建树。
搜索商品	按分类（递归子分类）、关键词、排序条件分页查询上架/预售商品。
查看详情	查询商品聚合信息（图/规格/SKU/评价），并累加浏览量。
查看推荐	排除当前商品，按同分类优先、热度排序取前 N 条。

本用例涉及的重要类与方法：服务接口 ICategoryService (GetTreeAsync)、IProductService (SearchPublicAsync、GetPublicDetailAndTrackAsync、GetRecommendationsAsync) 、仓储 ProductRepository（递归分类查询）。

2.3.2 用例实现

前台商品搜索在按分类过滤时使用 Oracle 递归查询展开全部启用的子分类，如代码 2-2 所示。

```
1 p."CATEGORY_ID" IN (  
2     SELECT c."ID"  
3     FROM "CATEGORY" c  
4     START WITH c."ID" = :categoryId AND c."STATUS" = 1  
5     CONNECT BY NOCYCLE PRIOR c."ID" = c."PARENT_ID" AND c."STATUS" = 1  
6 )
```

代码 2-2 ProductRepository 子分类递归查询

未登录访问首页、商品列表页与详情页分别如图 2-4、图 2-5 与图 2-6 所示。

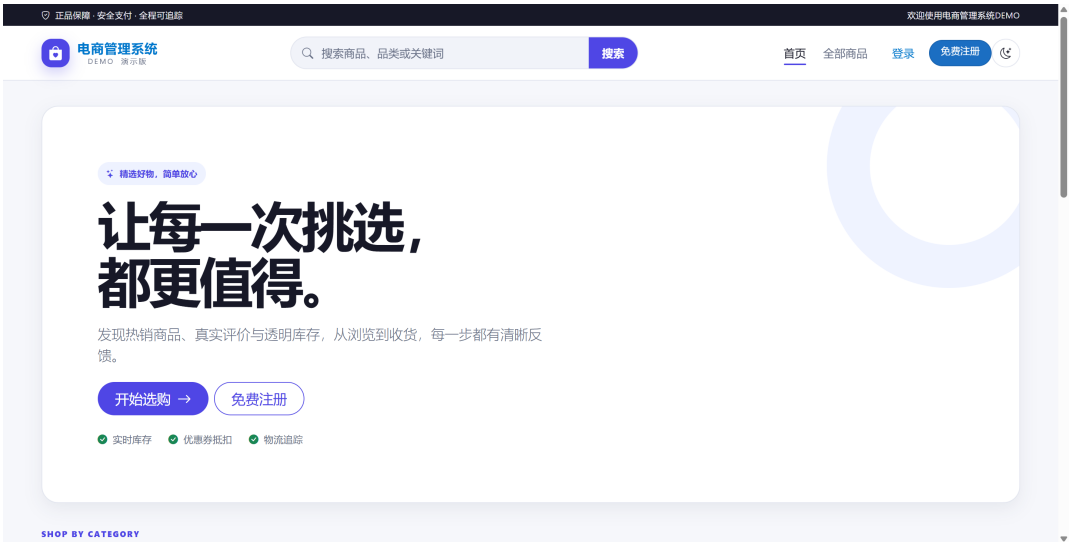


图 2-4 首页（未登录）

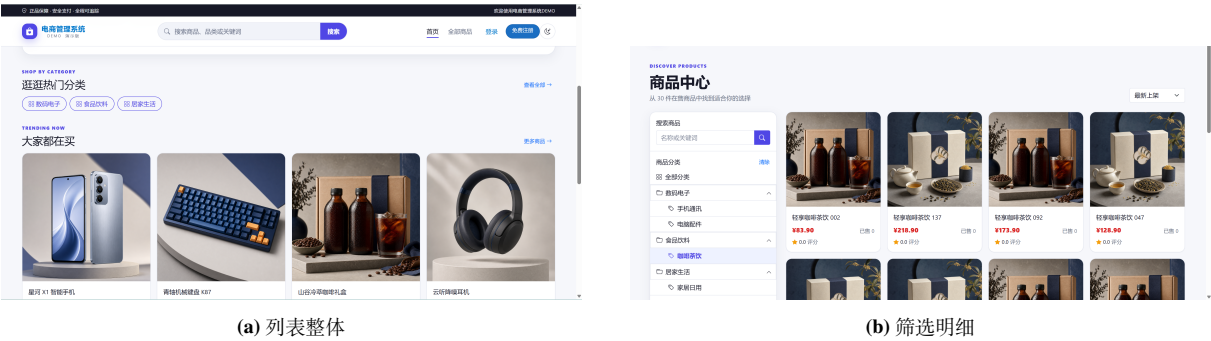


图 2-5 商品列表页面

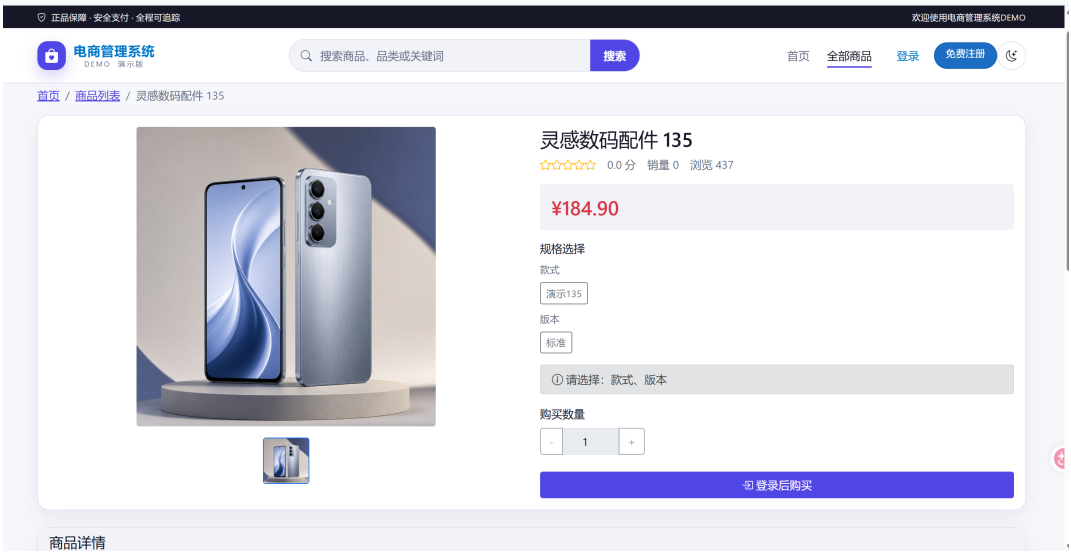


图 2-6 商品详情页面

2.4 购物车管理用例

2.4.1 用例设计

购物车管理用例面向登录用户，以 SKU 为单位维护购物车项。加入购物车时校验 SKU 在售状态与可用库存 (stock - locked_stock)，已存在的 SKU 通过原子累加数量避免并发丢失，首次加入利用唯一键 (user_id, sku_id) 冲突回退重试。购物车支持修改数量、勾选状态、删除与清空，结算仅统计勾选项并展示勾选金额合计。动作序列如表 2-3 所示。

表 2-3 购物车管理用例的动作序列

动作序列	描述
加入购物车	校验 SKU 在售与库存，原子累加已有项或新增，冲突时重试累加。
修改数量	校验归属与库存上限，覆盖数量与选中状态。
删除/清空	删除指定项或按用户一次清空。
查看购物车	联表查询商品快照与可用库存，计算勾选合计。

本用例涉及的重要类与方法：服务接口 ICartService (GetCartAsync、AddItemAsync、UpdateItemAsync、RemoveItemAsync、ClearAsync)、仓储 ICartRepository (TryIncreaseQuantityAsync 原子累加、AddAsync 捕获唯一键冲突)。

2.4.2 用例实现

加入购物车的核心并发控制如代码 2-3 所示：先尝试原子累加已有购物车项，插入时若遇到唯一键冲突（ORA-00001，两请求同时首次加入同一 SKU）则回退重试累加，避免超量。

```
1 var availableStock = sku.Stock - sku.LockedStock;
2 if (availableStock < request.Quantity)
3     throw new BusinessException("INSUFFICIENT_STOCK",
4         $"库存不足，当前可用库存：{availableStock}");
5
6 // 原子累加已有购物车项，避免“查询后更新”导致并发丢失。
7 if (await _cartRepository.TryIncreaseQuantityAsync(
8     userId, request.SkuId, request.Quantity,
9     availableStock, DateTime.Now, ct) == 1)
10    return;
11
12 var cart = new Cart { UserId = userId, SkuId = request.SkuId, ... };
13 try
14 {
15     await _cartRepository.AddAsync(cart, ct);
16     return;
17 }
18 catch (OracleException ex) when (ex.Number == 1)
19 {
20     // 并发首次加入同一 SKU 时，唯一键竞争失败方重试原子累加。
21     if (await _cartRepository.TryIncreaseQuantityAsync(
22         userId, request.SkuId, request.Quantity,
23         availableStock, DateTime.Now, ct) == 1)
24         return;
25 }
```

代码 2-3 CartService.AddItemAsync 并发加购

购物车页面如图 2-7 所示。

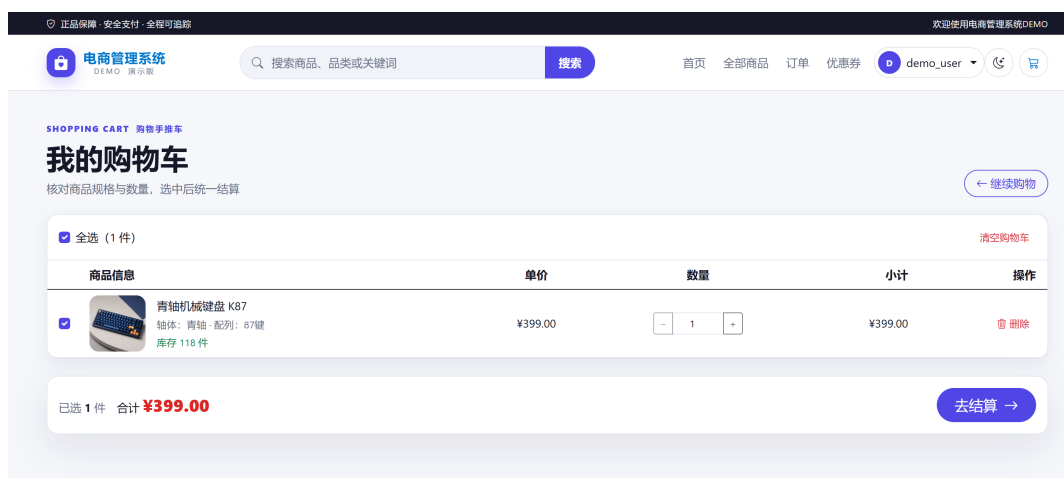


图 2-7 购物车页面

2.5 收货地址管理用例

2.5.1 用例设计

收货地址管理用例面向登录用户，是下单的前置环节。用户可新增、修改、删除收货地址，并设置默认地址；删除采用软删除，设置默认时在事务内先取消同用户旧默认地址、再设置新默认地址，配合数据库函数唯一索引保证一个用户至多一条有效默认地址。动作序列如表 2-4 所示。

表 2-4 收货地址管理用例的动作序列

动作序列	描述
新增地址	校验收货人、手机号等非空，写入地址；可指定为默认。
修改地址	校验地址归属后覆盖更新收货信息。
删除地址	软删除（置删除标记），默认地址同时取消默认标记。
设置默认	事务内先取消旧默认、再设置新默认，保证唯一默认。
下单取地址	校验地址归属，供订单收货人快照固化使用。

本用例涉及的重要类与方法：服务接口 `IAddressService` (`GetMyAddressesAsync`、`CreateAsync`、`UpdateAsync`、`DeleteAsync`、`SetDefaultAsync`) 、仓储 `AddressRepository` (`SetDefaultAsync` 事务内取消旧默认并设置新默认)。

2.5.2 用例实现

默认地址唯一性由数据库函数唯一索引从库层兜底，如代码 2-4 所示——该索引仅对「未删除且为默认」的地址取用户 ID 生成唯一键，从而保证任意时刻每个用户至多一条有效默认地址；服务层同时在事务内先取消旧默认、再设置新默认，双保险。

```
1 CREATE UNIQUE INDEX uk_address_one_default ON ADDRESS (  
2     CASE WHEN is_deleted = 0 AND is_default = 1 THEN user_id END  
3 );
```

代码 2-4 默认地址唯一性（函数唯一索引）

收货地址页面如图 2-8 所示。

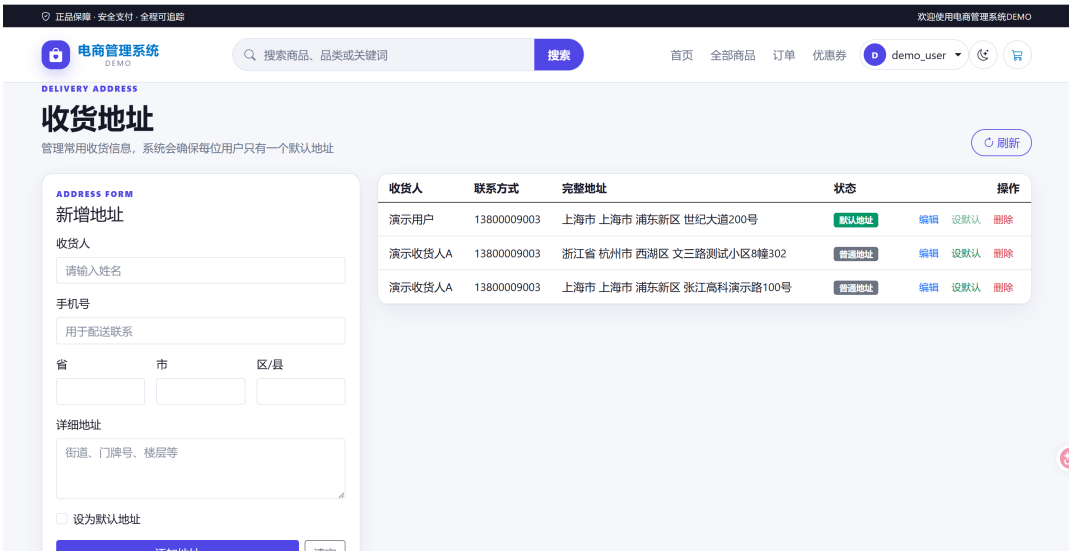


图 2-8 收货地址页面

2.6 订单创建用例

2.6.1 用例设计

订单创建是系统的核心用例，面向登录用户。用户从购物车勾选商品，选择收货地址与优惠券后提交订单。系统在服务端重新计算金额，并在一笔事务内完成：校验地址与 SKU 在售库存、再次验证优惠券、插入订单主表（含收货人 JSON 快照与 30 分钟支付超时时间）、插入订单明细（商品名/规格/主图/单价快照）、写订单状态日志、锁定库存、核销优惠券、清空购物车。任一环节失败整体回滚。动作序列如表 2-5 所示。

表 2-5 订单创建用例的动作序列

动作序列	描述
下单预览	校验地址、购物车项、SKU 库存，服务端试算金额与优惠。
创建订单	生成订单号，服务端重算金额并再次校验优惠券。
开启事务	插入订单主表、明细、日志；锁定库存；原子核销优惠券；清空购物车。
提交事务	全部成功则提交；任一步失败回滚，不留半成功数据。

本用例涉及的重要类与方法：服务接口 `IOrderService` (`PreviewAsync`、`CreateAsync`) 、`IInventoryService` (`LockForOrderAsync`) 、`ICouponService` (`UseForOrderAsync`) 、`IAddressService`，以及订单、购物车仓储与 `IUnitOfWork` 事务抽象。

2.6.2 用例实现

订单创建在事务内编排各子服务，核心代码如代码 2-5 所示（省略明细构造）：金额由服务端基于购物车重新计算，优惠券在订单事务内再次验证并原子核销，库存由库存服务锁定。

```
1  await _unitOfWork.BeginTransactionAsync(ct);

3  // 金额必须由服务端购物车重新计算，并在订单事务中再次验证优惠券。
4  var discountAmount = await GetDiscountAmountAsync(
5      userId, request.UserCouponId, totalAmount, categoryAmounts, ct);
6  var payAmount = Math.Max(0, totalAmount - discountAmount);
```

```
8 var order = new OrderMain {
9     OrderNo = orderNo, UserId = userId, AddressId = request.AddressId,
10    Status = (int)OrderStatus.PendingPayment,
11    TotalAmount = totalAmount, DiscountAmount = discountAmount,
12    PayAmount = payAmount,
13    PayExpireTime = DateTime.Now.AddMinutes(PAY_EXPIRE_MINUTES),
14    ReceiverSnapshot = JsonSerializer.Serialize(new {
15        address.ReceiverName, address.ReceiverPhone,
16        address.Province, address.City, address.District,
17        address.DetailAddress })
18 };
19 var orderId = await _orderRepository.InsertOrderMainAsync(order, ct);

21 if (request.UserCouponId.HasValue)
22     await _couponService.UseForOrderAsync(userId, request.UserCouponId.Value,
23         orderId, totalAmount, categoryAmounts, discountAmount, ct);

25 await _orderRepository.InsertOrderItemsAsync(items, ct);
26 await _orderRepository.InsertOrderLogAsync(new OrderLog { ... }, ct);

28 // 锁定库存 (可用库存 = stock - locked_stock)
29 var skuQuantities = items.Select(x => new OrderSkuQuantity(x.SkuId, x.Quantity)).ToList();
30 await _inventoryService.LockForOrderAsync(orderId, skuQuantities, ct);

32 if (request.CartItemIds is { Count: > 0 })
33     await _cartRepository.ClearByIdsAsync(userId,
34         cartItems.Select(i => i.CartItemId).ToArray(), ct);
35 else
36     await _cartRepository.ClearSelectedAsync(userId, ct);

38 await _unitOfWork.CommitAsync(ct);
39 return orderId;
```

代码 2-5 OrderService.CreateAsync 下单事务

结算页与订单提交成功后的支付页分别如图 2-9 与图 2-10 所示。

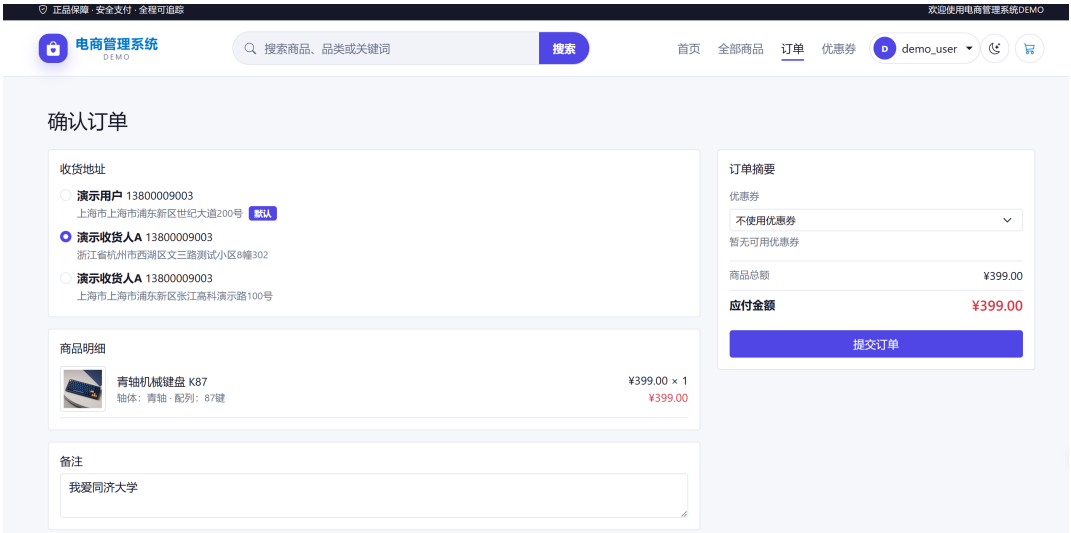


图 2-9 订单结算页面



图 2-10 支付页面

2.7 订单支付用例

2.7.1 用例设计

订单支付用例面向登录用户，系统采用持久化模拟支付通道。用户对待支付订单发起支付，系统在事务内写入支付记录并将订单推进为已支付、扣减锁定库存（deduct：实际库存与锁定库存同时减少）、写订单状态日志与库存流水；商品销量由数据库触发器 TRG_ORDER_PAID_UPDATE_SALES 在状态变更后自动累加。支付支持幂等：一订单一条支付记录（唯一约束），重复支付直接返回已成功。系统另提供模拟支付回调接口，以 HMAC-SHA256 签名（密钥来自环境变量）验证回调来源，防止匿名伪造。动作序列如表 2-6 所示。

表 2-6 订单支付用例的动作序列

动作序列	描述
创建支付单	幂等创建待支付记录，校验订单待支付且未超期。
发起模拟支付	事务内推进支付状态为成功、订单为已支付，扣减库存，写日志。
支付回调	HMAC 签名校验，幂等推进支付与订单状态。
完成支付	商品销量由触发器累加，支付单记录回调原文。

本用例涉及的重要类与方法：服务接口 `IPaymentService` (`CreateOrGetPendingAsync`、`SimulatePayAsync`、`SyncSimulatedCallbackAsync`)、`IOrderService` (`MarkPaidAsync`)、仓储 `PaymentRepository` (`TryMarkSuccessAsync` 条件更新)、`SkuRepository`.`DeductStockAsync`。

2.7.2 用例实现

库存扣减使用单条条件 `UPDATE` 原子完成，如代码 2-6 所示：仅当锁定库存足够时扣减实际库存与锁定库存，以影响行数判定并发竞争。

```
1 UPDATE "SKU"
2 SET "STOCK" = "STOCK" - :quantity,
3     "LOCKED_STOCK" = "LOCKED_STOCK" - :quantity,
4     "UPDATED_AT" = :updatedAt
5 WHERE "ID" = :skuId AND "LOCKED_STOCK" >= :quantity
```

代码 2-6 SkuRepository 库存扣减原语

模拟支付回调通过 `HMAC-SHA256` 签名校验来源，并使用 `CryptographicOperations.FixedTimeEquals` 恒定时间比较防止时序侧信道攻击，如代码 2-7 所示。

```
1 // 分隔符为竖线 (ASCII 124) , payload = orderIdtradeNostatuspayAmountrawData
2 var payload = string.Join((char)124,
3     request.OrderId.ToString(CultureInfo.InvariantCulture),
4     request.TradeNo.Trim(),
5     request.Status.ToString(CultureInfo.InvariantCulture),
6     request.PayAmount.ToString("0.00", CultureInfo.InvariantCulture),
7     request.RawData);
8 using var hmac = new HMACSHA256(
9     Encoding.UTF8.GetBytes(_options.SimulatedCallbackSecret));
10 var expected = Convert.ToHexString(
11     hmac.ComputeHash(Encoding.UTF8.GetBytes(payload)));
12 var actual = request.Signature.Trim().ToUpperInvariant();
13 if (actual.Length != expected.Length
14     !CryptographicOperations.FixedTimeEquals(
15         Encoding.ASCII.GetBytes(actual),
16         Encoding.ASCII.GetBytes(expected)))
17 {
18     throw new BusinessException("PAYMENT_SIGNATURE_INVALID",
19         "支付回调签名无效");
20 }
```

代码 2-7 PaymentService 模拟支付回调验签

支付页面如图 2-11 所示。

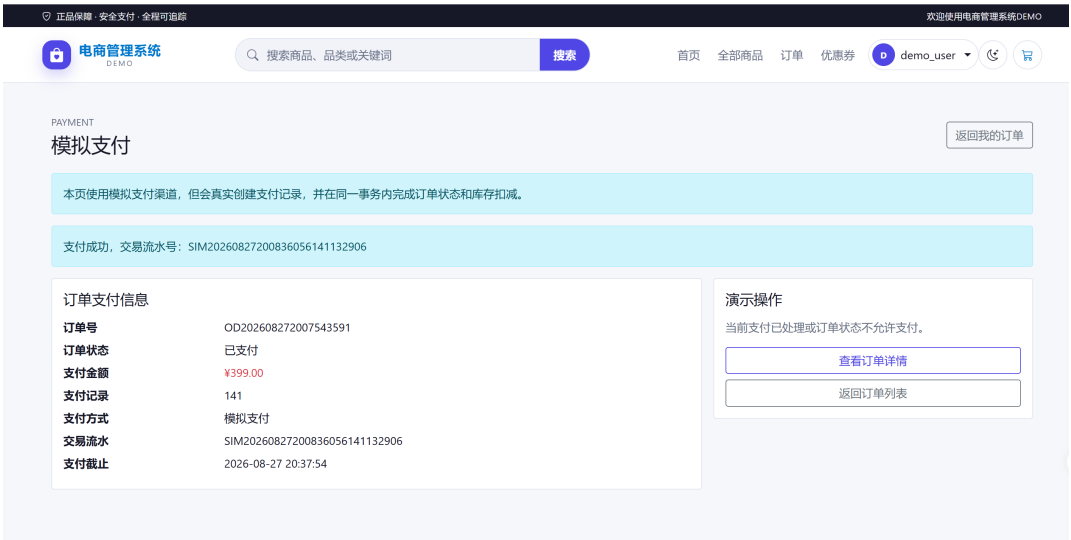


图 2-11 模拟支付页面

2.8 订单管理与超时关闭用例

2.8.1 用例设计

订单管理用例覆盖订单查询、取消、确认收货与超时自动关闭。用户可查询本人订单（分页、按状态/时间筛选），客服/管理员可查询后台订单（按订单号/用户/状态/时间筛选）。待支付订单可由用户主动取消或由后台取消；取消时在事务内将订单状态推进为已取消、释放锁定库存、回退优惠券，并区分“用户自取消”（仅写订单日志）与“后台取消”（额外写操作日志）。用户对已发货订单确认收货后订单完成。订单超时自动关闭由后台托管服务 `OrderTimeoutHostedService` 每分钟扫描待支付且超过 30 分钟未支付的订单，以系统账号走完整取消链路。所有状态流转均以“期望状态条件更新”实现，数据库触发器 `TRG_ORDER_STATUS_FLOW_GUARD` 兜底约束。动作序列如表 2-7 所示。

表 2-7 订单管理用例的动作序列

动作序列	描述
订单查询	我的订单/后台订单分页查询，按状态、时间、订单号、用户筛选。
取消订单	仅待支付可取消；事务内推进状态、释放锁定库存、回退优惠券。
确认收货	仅已发货订单可由所属用户确认，推进为已完成。
超时扫描	后台服务扫描超期待支付订单，逐单校验状态后自动取消。
状态保护	条件更新 + 触发器双保险，拒绝非法状态流转。

本用例涉及的重要类与方法：服务接口 `IOrderService` (`SearchMineAsync`、`SearchAdminAsync`、`GetDetailAsync`、`CancelAsync`、`ConfirmAsync`)、仓储 `IOrderRepository` (`SearchUserOrdersAsync`、`SearchAdminOrdersAsync`、`TryUpdateStatusAsync`、`GetExpiredOrderIdsAsync`)、托管服务 `OrderTimeoutHostedService`、`IInventoryService.ReleaseForCancelledOrderAsync`。

2.8.2 用例实现

订单状态推进采用期望状态条件更新，如代码 2-8 所示；取消订单时实付金额强制清零（数据库检查约束兜底）。其中 `:isCancelled` 由仓储层在 `TryUpdateStatusAsync` 内部依据目标状态是否为已取消（4）计算后传入。

```
1 UPDATE order_main
2 SET status = :targetStatus,
3     updated_at = :updatedAt,
4     pay_amount = CASE WHEN :isCancelled = 1 THEN 0 ELSE pay_amount END
5 WHERE id = :orderId AND status = :expectedStatus
```

代码 2-8 OrderRepository 期望状态条件更新

“我的订单” 页面如图 2-12 所示，后台订单管理页面如图 2-13 所示。

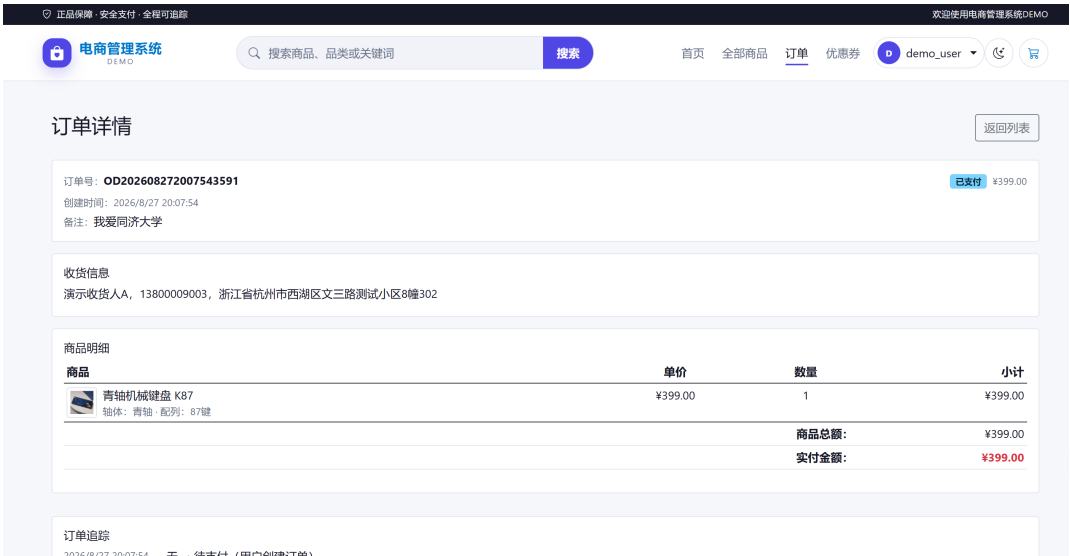


图 2-12 我的订单页面

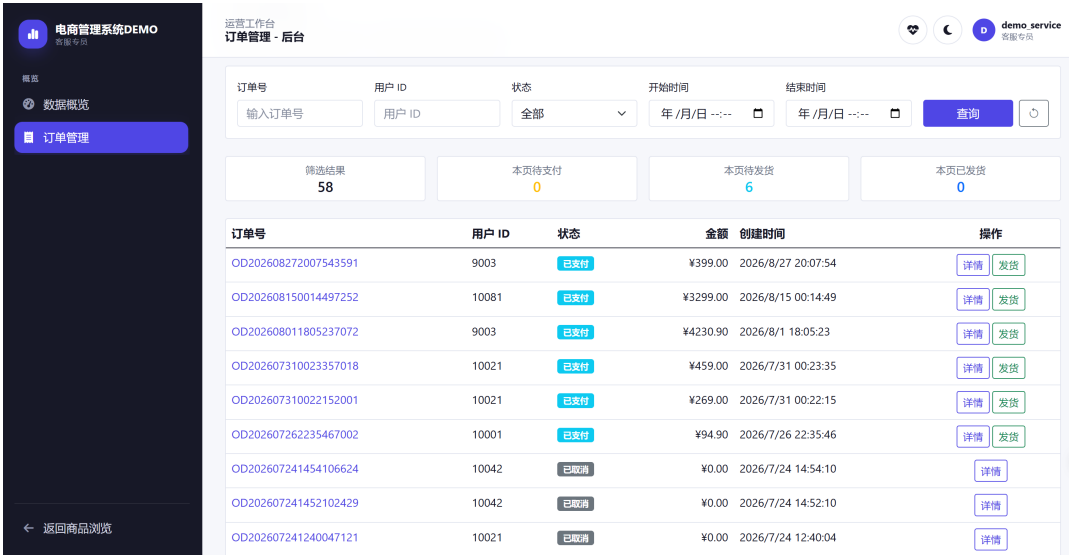


图 2-13 后台订单管理页面

2.9 优惠券领取与使用用例

2.9.1 用例设计

优惠券用例面向用户与管理员。管理员创建满减（类型 1）或折扣（类型 2）券模板，可限定适用分类（必须为启用的末级分类）；用户可领取启用且在有效期、未领完的券，领取时通过原子自增 `received_count` 防超发，并以唯一键（`user_id`, `coupon_template_id`）防重复领取。下单时校验券的状态、有效期、门槛与适用分类金额维度；核销采用单条条件 `UPDATE`，同时校验服务端重算的期望折扣，防止前端篡改；订单取消时回退优惠券。动作序列如表 2-8 所示。

表 2-8 优惠券领取与使用用例的动作序列

动作序列	描述
创建模板	后台创建满减/折扣模板，校验发行量与适用分类。
领取优惠券	原子自增领取计数防超发，唯一键防重复领取。
校验优惠券	校验状态、有效期、门槛、适用分类金额，计算折扣。
核销优惠券	下单事务内条件更新为已使用，校验期望折扣防篡改。
回退优惠券	订单取消时恢复券为未使用。

本用例涉及的重要类与方法：服务接口 `ICouponService`（`ReceiveAsync`、`ValidateAsync`、`UseForOrderAsync`、`RestoreForOrderAsync`）、仓储 `CouponRepository`（`TryIncrementReceivedCountAsync`、`TryUseForOrderAsync`）。

2.9.2 用例实现

优惠券核销通过单条 `UPDATE` 原子完成，并在 `EXISTS` 子查询中重新校验模板状态、有效期、门槛与折扣计算（与服务端期望折扣一致），如代码 2-9 所示。

```
1 UPDATE user_coupon
2 SET status = :UsedStatus, used_at = :UsedAt, order_id = :OrderId
3 WHERE id = :UserCouponId
4     AND user_id = :UserId
5     AND status = :UnusedStatus
6     AND EXISTS (
7         SELECT 1 FROM coupon_template ct
8         WHERE ct.id = user_coupon.coupon_template_id
9             AND ct.status = 1
10            AND ct.start_time <= :UsedAt AND ct.end_time >= :UsedAt
11            AND ct.min_amount <= :EligibleAmount
12            AND ((ct.type = 1 AND ct.amount > 0 AND ct.amount <= :EligibleAmount
13                AND ct.amount = :ExpectedDiscountAmount)
14            OR (ct.type = 2 AND ct.amount > 0 AND ct.amount <= 1
15                AND ROUND(:EligibleAmount * (1 - ct.amount), 2) = :ExpectedDiscountAmount)))
```

代码 2-9 CouponRepository 优惠券原子核销

优惠券中心页面如图 2-14 所示。

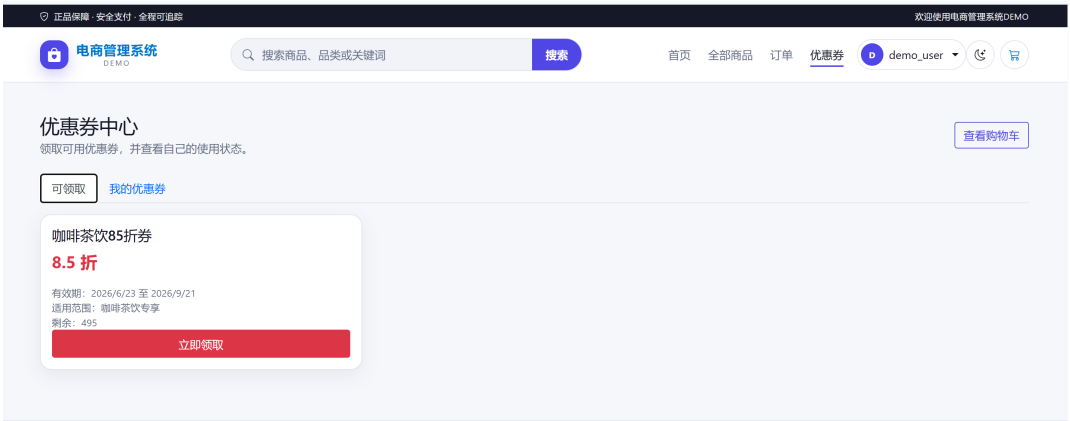


图 2-14 优惠券中心页面

2.10 物流发货与轨迹用例

2.10.1 用例设计

物流用例面向客服（SERVICE）与用户。客服对已支付订单发货：填写物流公司与运单号，在事务内创建物流记录（状态为已揽收）、写入首条轨迹、调用订单服务将订单推进为已发货、写订单与操作日志；物流记录与运单号均唯一，重复发货会触发唯一约束冲突并被转译为友好错误。客服可追加物流轨迹，轨迹状态只允许向前推进（不可回退）。用户可查询本人订单的物流与轨迹。动作序列如表 2-9 所示。

表 2-9 物流发货与轨迹用例的动作序列

动作序列	描述
发货	校验订单已支付；事务内建物流单、首条轨迹，推进订单为已发货。
追加轨迹	校验状态不回退，插入轨迹并原子推进物流状态。
查询物流	用户校验订单归属后查询物流与轨迹；后台直接查询。

本用例涉及的重要类与方法：服务接口 `ILogisticsService`（`ShipAsync`、`AddTrackAsync`、`GetByOrderAsync`）、`IOrderService`（`MarkShippedAsync`）、仓储 `ILogisticsRepository`（`TryUpdateStatusAsync` 状态推进）。

2.10.2 用例实现

发货在同一事务内创建物流记录、写入首条轨迹并推进订单状态，如代码 2-10 所示；唯一键冲突（重复发货）被捕获并转译为业务错误。

```
1  await _unitOfWork.BeginTransactionAsync(ct);
2  try
3  {
4      var logistics = new Logistics { OrderId = orderId,
5          CompanyName = request.CompanyName.Trim(),
6          TrackingNo = request.TrackingNo.Trim(),
7          ShippedAt = shippedAt,
8          Status = (int)LogisticsStatus.Collectected };
9      var logisticsId = await _logisticsRepository.InsertAsync(logistics, ct);
10     await _logisticsRepository.InsertTrackAsync(new LogisticsTrack {
```



```
11     LogisticsId = logisticsId,
12     TrackDesc = "商家已发货，快件已揽收",
13     TrackTime = shippedAt }, ct);

15     await _orderService.MarkShippedAsync(orderId, logisticsId,
16         operatorId, operatorName, ipAddress, ct);
17     await _unitOfWork.CommitAsync(ct);
18 }
19 catch (Exception exception)
20 {
21     await _unitOfWork.RollbackAsync(ct);
22     if (exception is OracleException { Number: 1 })
23         throw new BusinessException("LOGISTICS_ALREADY_EXISTS",
24             "该订单已有物流信息或运单号已存在");
25     throw;
26 }
```

代码 2-10 LogisticsService.ShipAsync 发货事务

订单物流页面如图 2-15 所示。



图 2-15 订单物流页面

2.11 商品评价用例

2.11.1 用例设计

商品评价用例面向已完成订单的用户。用户对已完成且包含该商品的订单提交评价（评分 1~5、文字、最多 9 张图片、可匿名），系统校验订单归属、订单完成状态、商品属于该订单并防止重复评价（应用校验 + 唯一约束（order_id, product_id, user_id）兜底）。评价初始为“待审核”，管理员审核发布或屏蔽；前台按商品展示已发布评价并对匿名评价隐藏用户 ID，审核状态变更时同步重算商品评分均值。动作序列如表 2-10 所示。

表 2-10 商品评价用例的动作序列

动作序列	描述
提交评价	校验订单完成、归属、含商品、未重复评价，写入待审核评价。
审核评价	管理员发布或屏蔽：发布后对前台可见并重算商品均分。
查看评价	前台按商品展示已发布评价，匿名评价隐藏用户。

本用例涉及的重要类与方法：服务接口 IReviewService (CreateAsync、SearchByProductAsync、SetStatusAsync)、仓储 IReviewRepository (OrderContainsProductAsync、RefreshProductAverageRatingAsync)。

2.11.2 用例实现

评价提交前的完整业务校验如代码 2-11 所示，插入时捕获唯一约束冲突防止并发重复评价。

```
1 var order = await _orderRepository.GetByIdAsync(request.OrderId, ct)
2     ?? throw new BusinessException("ORDER_NOT_FOUND", "订单不存在");
3 if (order.UserId != userId)
4     throw new BusinessException("FORBIDDEN", "不能评价其他用户的订单");
5 if (order.Status != (int)OrderStatus.Completed)
6     throw new BusinessException("ORDER_NOT_COMPLETED", "订单完成后才能评价商品");
7 if (!await _reviewRepository.OrderContainsProductAsync(
8     request.OrderId, request.ProductId, ct))
9     throw new BusinessException("PRODUCT_NOT_IN_ORDER", "订单中不包含该商品");
10 if (await _reviewRepository.HasReviewedAsync(
11     request.OrderId, request.ProductId, userId, ct))
12     throw new BusinessException("ALREADY_REVIEWED", "该订单商品已经评价");

14 try { return await _reviewRepository.InsertAsync(review, ct); }
15 catch (OracleException exception) when (exception.Number == 1)
16 { throw new BusinessException("ALREADY_REVIEWED", "该订单商品已经评价"); }
```

代码 2-11 ReviewService.CreateAsync 评价校验

商品详情页的评价区如图 2-16 所示。

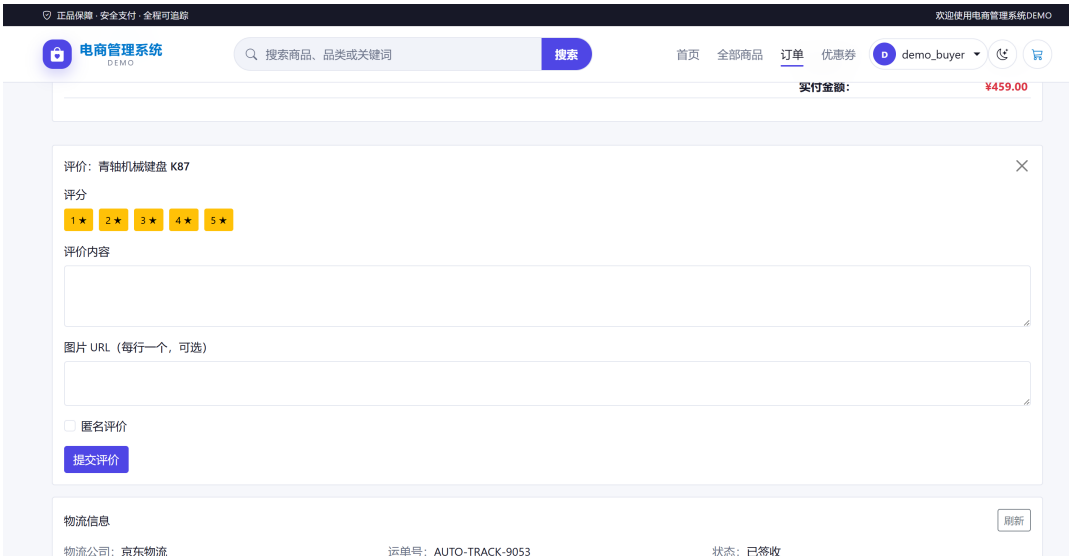


图 2-16 商品评价区

2.12 后台商品与库存管理用例

2.12.1 用例设计

后台商品与库存管理用例面向管理员（ADMIN），是数据维护的核心。管理员维护分类树（防止形成分类循环引用、层级校验、存在子分类或商品时拒绝删除）、商品（复合保存：商品主表、图片、规格、SKU 一次提交，SKU 规格必须选择齐全且组合唯一）、库存（手工调整、低库存预警、库存变动流水）。库存调整在事务内更新 SKU 并写库存流水（ADJUST），校验调整后库存非负且不低于锁定库存。动作序列如表 2-11 所示。

表 2-11 后台商品与库存管理用例的动作序列

动作序列	描述
维护分类	增改分类，校验层级与防止成环；有子分类或商品时拒绝删除。
维护商品	复合保存商品、图片、规格、SKU；SKU 规格组合唯一。
调整库存	事务内更新 SKU 库存并写库存流水，校验库存下限。
预警与流水	按可用库存阈值过滤预警，按条件分页查询库存流水。

本用例涉及的重要类与方法：服务接口 ICategoryService、IProductService、ISkuService、IInventoryService（AdjustAsync、SearchWarningsAsync）、IProductImageService。

2.12.2 用例实现

库存手工调整在事务内更新 SKU 并写入库存流水（ADJUST），校验调整后库存非负且不低于锁定库存，如代码 2-12 所示。

```
1  await _unitOfWork.BeginTransactionAsync(ct);
2  var sku = await _skuRepository.GetByIdAsync(skuId, ct);
3  var beforeStock = sku.Stock;
4  var afterStock = sku.Stock + request.ChangeQty;

6  // 调整后库存不能为负，且不能低于锁定库存
7  if (afterStock < 0  afterStock < sku.LockedStock)
```

```
8      throw new BusinessException("INVENTORY_INSUFFICIENT",
9          "库存不足，调整后库存不能小于锁定库存");

11     sku.Stock = afterStock;
12     await _skuRepository.UpdateAsync(sku, ct);

14     // 写库存变动流水 (ADJUST)，记录调整前后库存
15     await _inventoryLogRepository.CreateAsync(new InventoryLog {
16         SkuId = skuId, ChangeType = InventoryChangeType.Adjust,
17         ChangeQty = request.ChangeQty, BeforeStock = beforeStock,
18         AfterStock = afterStock, OperatorId = operatorId,
19         Remark = request.Remark }, ct);

21     await _unitOfWork.CommitAsync(ct);
```

代码 2-12 InventoryService.AdjustAsync 库存手工调整

后台商品管理页与库存预警页分别如图 2-17 与图 2-18 所示。



图 2-17 后台商品管理页面

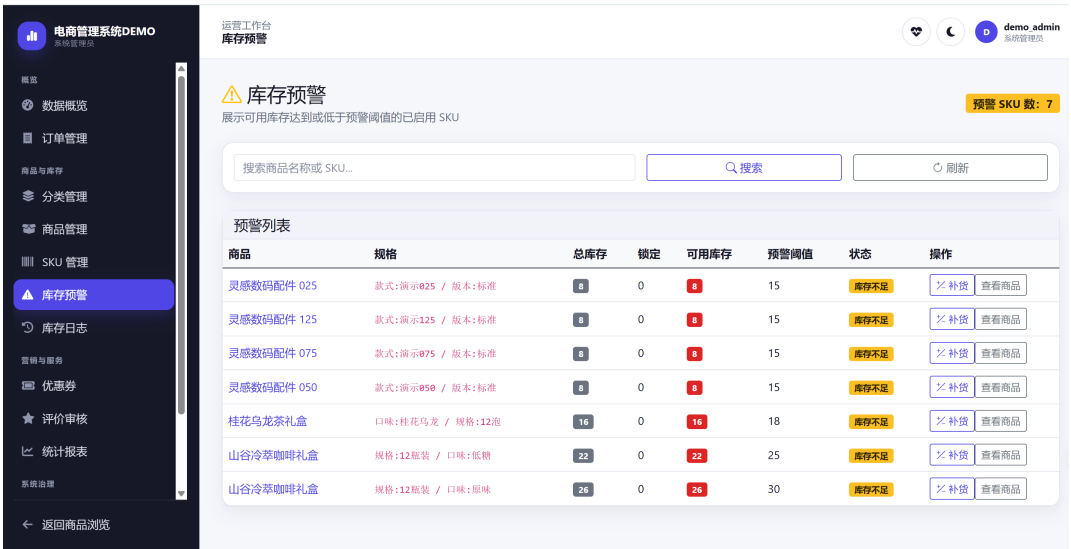


图 2-18 后台库存预警页面

2.13 后台权限、统计与审计用例

2.13.1 用例设计

后台权限、统计与审计用例面向管理员与客服。权限控制采用 RBAC 模型：用户—角色、角色—权限通过关联表多对多绑定，系统预置 ADMIN（管理员）、SERVICE（客服）、USER（用户）三角色。授权在端点级策略（CustomerOnly/ServiceOrAdmin/AdminOnly）之上叠加动态 RBAC 过滤器：按请求路径与 HTTP 方法匹配权限规则（支持 * 单段与 ** 多段通配），ADMIN 恒放行，严格后台路径默认拒绝。管理员保护：不能禁用自己、不能移除自己的 ADMIN 角色、系统始终保留至少一个启用管理员。统计采用快照表 + 定时回填，后台关键写操作全部写入操作日志。动作序列如表 2-12 所示。

表 2-12 后台权限、统计与审计用例的动作序列

动作序列	描述
请求鉴权	端点级策略过滤；动态 RBAC 按路径与方法匹配权限规则。
角色权限绑定	管理员为角色全量绑定权限，ADMIN 角色权限只读。
统计与导出	快照表聚合统计，订单/库存流水导出 Excel（上限 5000 行）。
操作审计	后台关键写操作写操作日志（模块/动作/IP/结果）。

本用例涉及的重要类与方法：服务接口 IPermissionService (CanAccessAsync、BindRolePermissionsAsync)、IStatisticsService、IExportService、IOperationLogService、Web 层过滤器 RbacPermissionFilter 与 AdminOperationAuditFilter。

2.13.2 用例实现

动态 RBAC 的核心逻辑如代码 2-13 所示：ADMIN 恒放行，无匹配规则时仅对严格后台路径拒绝，有匹配规则时按角色的权限规则判定，规则经内存缓存加速。

```
1 if (roleNames.Contains(AuthConstants.Roles.Admin, StringComparer.OrdinalIgnoreCase))
2     return true;
```

```
4 var action = httpMethod.ToUpperInvariant();
5 var allRules = await _cache.GetOrCreateAsync(GetAllRulesCacheKey(action), ...)
6     ?? Array.Empty<PermissionDto>();
7 var matchingRules = allRules
8     .Where(rule => PermissionPathMatcher.IsMatch(rule.ResourcePath, requestPath))
9     .ToArray();
10 if (matchingRules.Length == 0)
11     return !IsStrictBackendPath(requestPath);
13 foreach (var roleName in roleNames.Select(r => r.Trim()).Distinct(...))
14 {
15     var roleRules = await _cache.GetOrCreateAsync(
16         GetRoleRulesCacheKey(roleName, action), ...) ?? Array.Empty<PermissionDto>();
17     if (roleRules.Any(rule => PermissionPathMatcher.IsMatch(rule.ResourcePath, requestPath))
18     )
19         return true;
20 }
return false;
```

代码 2-13 PermissionService.CanAccessAsync 动态鉴权

后台工作台（统计）页面与操作日志页面分别如图 2-19 与图 2-20 所示。

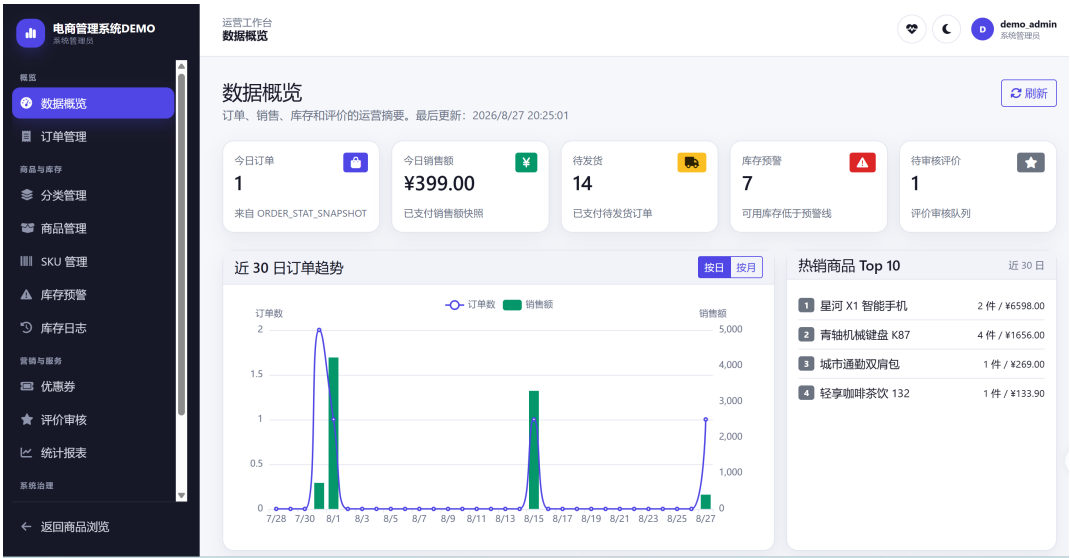


图 2-19 后台工作台页面

电商管理系统DEMO

系统管理

系统管理

数据概览

订单管理

商品与库存

分类管理

商品管理

SKU 管理

库存预警

库存日志

营销与推广

优惠券

评价审核

统计报表

系统设置

返回商品浏览

运营工作台

库存日志

库存日志

记录库存变动，支持按商品、SKU 和变更类型查询

导出 Excel

搜索商品名称或 SKU...

全部类型

SKU ID (可选)

搜索

重置

变动记录

共 188 条记录

日志ID	SKU	商品	类型	变更数量	变更前	变更后	操作人	时间	备注
108	9003	青轴机械键盘 K87	订单扣减	-1	118	117	系统	2026/8/27 20:08:36	订单支付扣减库存: 141
107	9003	青轴机械键盘 K87	订单锁定	+1	118	118	系统	2026/8/27 20:07:55	订单锁定库存: 141
88	9001	星河 X1 智能手机	订单扣减	-1	79	78	系统	2026/8/15 00:15:57	订单支付扣减库存: 121
87	9001	星河 X1 智能手机	订单锁定	+1	79	79	系统	2026/8/15 00:14:49	订单锁定库存: 121
84	9232	轻享咖啡茶饮 132	订单扣减	-1	64	63	系统	2026/8/1 18:11:05	订单支付扣减库存: 101
86	9003	青轴机械键盘 K87	订单扣减	-2	120	118	系统	2026/8/1 18:11:05	订单支付扣减库存: 101
85	9001	星河 X1 智能手机	订单扣减	-1	80	79	系统	2026/8/1 18:11:05	订单支付扣减库存: 101
81	9232	轻享咖啡茶饮 132	订单锁定	+1	64	64	系统	2026/8/1 18:05:23	订单锁定库存: 101
83	9003	青轴机械键盘 K87	订单锁定	+2	120	120	系统	2026/8/1 18:05:23	订单锁定库存: 101
82	9001	星河 X1 智能手机	订单锁定	+1	80	80	系统	2026/8/1 18:05:23	订单锁定库存: 101

图 2-20 后台操作日志页面

3 数据库设计

系统采用 Oracle 数据库，共设计 24 张业务表。主键统一使用自增 ID（GENERATED BY DEFAULT AS IDENTITY，其中用户表主键自 10001 起），金额字段使用 NUMBER(10,2)（统计快照表的销售额、退款金额为 NUMBER(12,2)），时间字段使用 DATE，长文本使用 CLOB。关键业务规则（订单状态流转、支付金额一致、地址归属、支付后销量累加）通过数据库触发器与检查约束在库层强制保证。

3.1 E-R 图

系统总体 E-R 图如图 3-1 所示，覆盖用户、商品、库存、订单、支付、物流、营销等全部 22 个概念实体及其联系。用户—角色、角色—权限为多对多联系，由中间表实现；订单一支付、订单一物流为一对一联系。概念图含 22 个实体，物理表共 24 张——两者差值为用户—角色、角色—权限两张多对多关联表（概念图中以菱形联系表达，物理模型展开为独立表）。

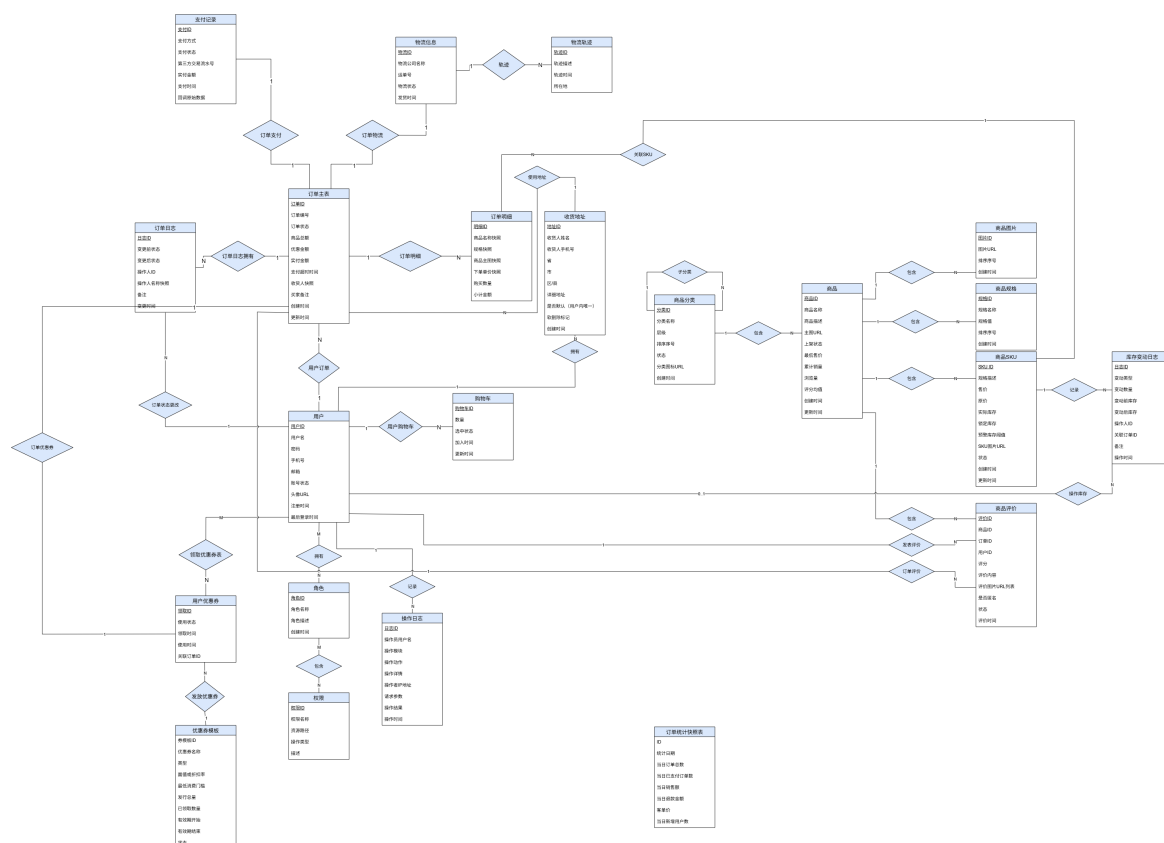


图 3-1 系统总体 E-R 图

各业务模块（用户与权限、商品与库存、订单与支付、营销与统计）的 E-R 图与总体图一致，分别描述各模块内部实体与联系，详见《数据库设计文档》，本文不再重复列出。

3.2 关系图

系统数据库关系图（物理模型）如图 3-2 所示，完整列出 24 张表的字段与外键连接关系，与建库脚本 `migration/init_database.sql` 一致。

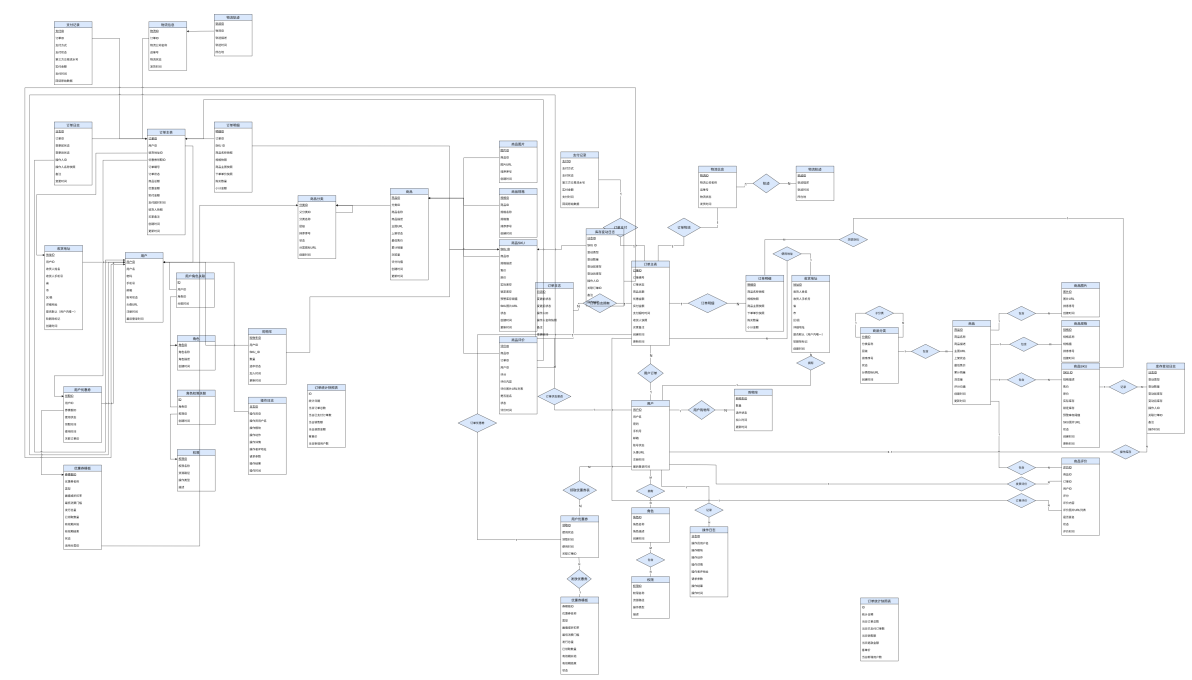


图 3-2 数据库关系图

3.3 关系模式说明

以下按模块给出全部 24 张表的关系模式（表名与字段均与建库脚本一致，下划线标注主键）。

表名	关系模式（下划线为主键）	主键 / 外键 / 约束
(1) 用户与权限模块		
USER	<u>id</u> , username, password_hash, phone, email, avatar_url, status, created_at, last_login_at	UQ: username、phone、email
ROLE	<u>id</u> , role_name, description, created_at	UQ: role_name
PERMISSION	<u>id</u> , perm_name, resource_path, action, description	UQ: perm_name
USER_ROLE	<u>id</u> , user_id, role_id, assigned_at	FK: user_id→USER、role_id→ROLE; UQ: (user_id, role_id)
ROLE_PERMISSION	<u>id</u> , role_id, permission_id, created_at	FK: role_id→ROLE、 permission_id→PERMISSION; UQ: (role_id, permission_id)
ADDRESS	<u>id</u> , user_id, receiver_name, receiver_phone, province, city, district, detail_address, is_default, is_deleted, created_at	FK: user_id→USER; 函数唯一索引保证 每用户一个默认地址
OPERATION_LOG	<u>id</u> , operator_id, operator_name, module, action, description, ip_address, request_params, result, created_at	FK: operator_id→USER
(2) 商品与库存模块		
CATEGORY	<u>id</u> , parent_id, name, tree_level, sort_order, status, icon_url, created_at	FK: parent_id→CATEGORY（自引用）; 层级 1 5
PRODUCT	<u>id</u> , category_id, name, description, main_image, status, price_min, sales_count, view_count, avg_rating, created_at, updated_at	FK: category_id→CATEGORY
PRODUCT_IMAGE	<u>id</u> , product_id, image_url, sort_order, created_at	FK: product_id→PRODUCT

表名	关系模式（下划线为主键）	主键 / 外键 / 约束
PRODUCT_SPEC	<u>id</u> , product_id, spec_name, spec_value, sort_order, created_at	FK: product_id→PRODUCT
SKU	<u>id</u> , product_id, spec_desc, price, original_price, stock, locked_stock, warning_stock, sku_image, status, created_at, updated_at	FK: product_id→PRODUCT; UQ: (product_id, spec_desc); CHECK: stock≥locked_stock≥0、spec_desc 为 JSON
INVENTORY_LOG	<u>id</u> , sku_id, change_type, change_qty, before_stock, after_stock, operator_id, ref_order_id, remark, created_at	FK: sku_id→SKU、operator_id→USER; ref_order_id 无外键（逻辑关联订单）
CART	<u>id</u> , user_id, sku_id, quantity, selected, created_at, updated_at	FK: user_id→USER、sku_id→SKU; UQ: (user_id, sku_id); CHECK: quantity≥1
(3) 订单与支付模块		
ORDER_MAIN	<u>id</u> , order_no, user_id, address_id, user_coupon_id, status, total_amount, discount_amount, pay_amount, pay_expire_time, receiver_snapshot, remark, created_at, updated_at	FK: user_id→USER、address_id→ADDRESS、user_coupon_id→USER_COUPON（可空）; UQ: order_no; CHECK: 实付 = 总额 - 优惠、取消单实付 0
ORDER_ITEM	<u>id</u> , order_id, sku_id, product_name_snap, spec_snap, main_image_snap, unit_price, quantity, subtotal	FK: order_id→ORDER_MAIN、sku_id→SKU; CHECK: subtotal=unit_price×quantity
ORDER_LOG	<u>id</u> , order_id, from_status, to_status, operator_id, operator_name, remark, created_at	FK: order_id→ORDER_MAIN、operator_id→USER
PAYMENT	<u>id</u> , order_id, pay_method, status, trade_no, pay_amount, paid_at, callback_data	FK: order_id→ORDER_MAIN（唯一）
LOGISTICS	<u>id</u> , order_id, company_name, tracking_no, shipped_at, status	FK: order_id→ORDER_MAIN（唯一）; UQ: tracking_no
LOGISTICS_TRACK	<u>id</u> , logistics_id, track_desc, track_time, location	FK: logistics_id→LOGISTICS
REVIEW	<u>id</u> , order_id, product_id, user_id, rating, content, images, is_anonymous, status, created_at	FK: order_id→ORDER_MAIN、product_id→PRODUCT、user_id→USER; UQ: (order_id, product_id, user_id); CHECK: rating 1 5
(4) 营销模块		
COUPON_TEMPLATE	<u>id</u> , name, type, amount, min_amount, total_count, received_count, start_time, end_time, status, applicable_category_id	FK: applicable_category_id→CATEGORY（可空，NULL 表示全场通用）; type 1 满减/2 折扣
USER_COUPON	<u>id</u> , user_id, coupon_template_id, status, received_at, used_at, order_id	FK: user_id→USER、coupon_template_id→COUPON_TEMPLATE; order_id 无外键; UQ: (user_id, coupon_template_id)
(5) 统计模块		
ORDER_STAT_SNAPSHOT	<u>id</u> , stat_date, order_count, paid_count, sales_amount, refund_amount, avg_order_amount, new_user_count	UQ: stat_date; 存储过程按日刷新

除上述表结构外，数据库还定义了函数 FN_AVAILABLE_STOCK（返回 SKU 可用库存）、存储过程 SP_REFRESH_ORDER_STAT_SNAPSHOT（按日刷新统计快照）、视图 V_PRODUCT_INVENTORY 与 V_ORDER_REPORT（供库存预警与订单导出查询），以及四个触发

器：TRG_ORDER_ADDRESS_OWNER_GUARD（订单地址归属校验） 、 TRG_PAYMENT_AMOUNT_GUARD（支付金额一致校验） 、 TRG_ORDER_STATUS_FLOW_GUARD（订单状态合法流转约束） 、 TRG_ORDER_PAID_UPDATE_SALES（订单推进为已支付后自动累加商品销量）。

A 图表索引

本文档所包含的图表索引如下。

编号	名称	页码
图 2-1	系统分层架构与请求流转	3
图 2-2	用户登录页面	5
图 2-3	用户注册页面	5
图 2-4	首页（未登录）	6
图 2-5	商品列表页面	7
图 2-6	商品详情页面	7
图 2-7	购物车页面	8
图 2-8	收货地址页面	10
图 2-9	订单结算页面	12
图 2-10	支付页面	12
图 2-11	模拟支付页面	14
图 2-12	我的订单页面	15
图 2-13	后台订单管理页面	15
图 2-14	优惠券中心页面	17
图 2-15	订单物流页面	18
图 2-16	商品评价区	20
图 2-17	后台商品管理页面	21
图 2-18	后台库存预警页面	22
图 2-19	后台工作台页面	23
图 2-20	后台操作日志页面	24
图 3-1	系统总体 E-R 图	25
图 3-2	数据库关系图	26
表 2-1	用户注册与登录用例的动作序列	3
表 2-2	商品浏览与搜索用例的动作序列	6
表 2-3	购物车管理用例的动作序列	7
表 2-4	收货地址管理用例的动作序列	9
表 2-5	订单创建用例的动作序列	10
表 2-6	订单支付用例的动作序列	12
表 2-7	订单管理用例的动作序列	14
表 2-8	优惠券领取与使用用例的动作序列	16
表 2-9	物流发货与轨迹用例的动作序列	17
表 2-10	商品评价用例的动作序列	19
表 2-11	后台商品与库存管理用例的动作序列	20
表 2-12	后台权限、统计与审计用例的动作序列	22
代码 2-1	AuthService.LoginAsync 登录校验	4
代码 2-2	ProductRepository 子分类递归查询	6
代码 2-3	CartService.AddItemAsync 并发加购	8
代码 2-4	默认地址唯一性（函数唯一索引）	9
代码 2-5	OrderService.CreateAsync 下单事务	10
代码 2-6	SkuRepository 库存扣减原语	13
代码 2-7	PaymentService 模拟支付回调验签	13

编号	名称	页码
代码 2-8	OrderRepository 期望状态条件更新	15
代码 2-9	CouponRepository 优惠券原子核销	16
代码 2-10	LogisticsService.ShipAsync 发货事务	17
代码 2-11	ReviewService.CreateAsync 评价校验	19
代码 2-12	InventoryService.AdjustAsync 库存手工调整	20
代码 2-13	PermissionService.CanAccessAsync 动态鉴权	22